

Lab - 1:

Experiment No: 1

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. **Hash Generation using SHA-256:**
 - o Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.
2. **Target Hash Generation with Nonce:**
 - o Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:**
 - o Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. **Merkle Tree Construction:**
 - o Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output

Program 1: Python program that uses the hashlib library to create the hash of a given string:

```
import hashlib
def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()
    # Return the hash string
    return hash_string
# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

Output:

Enter a string: KHUSHI SHUKLA

Hash: bb96cca811fcb0b5c1f283d736166097a97e519298113527ad3c15618efd0d7

Program 2: Program to generate required target hash with input string and nonce

```
import hashlib
# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")
# Concatenate the string and nonce
hash_string = input_string + nonce
# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()
# Print the hash code
```

```
print("Hash Code:", hash_code)
```

Output:

Enter a string: khushi shukla

Enter the nonce: 40

Hash Code: 014b8f48e2f194bb54f85c45ec70ba0fcc9f09ad0c057411a7ed49c3cdc37d48

Program 3: Python code for Solving Puzzle for leading zeros with expected nonce and given string

```
import hashlib
def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros
    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()
        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:", hash_code)
            return nonce
        nonce += 1
# Get user input
input_string = "khushi shukla"
num_zeros = 1
# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)
# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
```

Output:

Hash: 014b8f48e2f194bb54f85c45ec70ba0fcc9f09ad0c057411a7ed49c3cdc37d48

Input String: khushi shukla

Leading Zeros: 1

Expected Nonce: 40

Program 4: Generating Merkle Tree for given set of Transactions

```
import hashlib

def sha256(data):

    return hashlib.sha256(data.encode("utf-8")).hexdigest()

def build_merkle_tree(transactions):

    # STEP 1: Check if transactions are empty

    if len(transactions) == 0:

        print("No transactions!")

        return None

    print("===== STEP 1: ORIGINAL TRANSACTIONS =====")

    for i, transaction in enumerate(transactions, 1):

        print(f"Transaction {i}: {transaction}")

    # Make a copy so the original transactions are not changed

    current_level = transactions.copy()

    # STEP 2: Continue until only one hash remains

    step = 2

    while len(current_level) > 1:

        print(f"\n===== STEP {step}: CURRENT LEVEL =====")

        for i, item in enumerate(current_level, 1):

            print(f"{i}. {item}")

        # STEP 3: If number of items is odd, duplicate the last item

        if len(current_level) % 2 != 0:
```

```

print("\nNumber of items is ODD.")

print("Duplicating the last item...")

current_level.append(current_level[-1])

print(f"Duplicated: {current_level[-1]}")

# STEP 4: Combine pairs and hash them

print("\n===== HASHING PAIRS =====")

new_level = []

for i in range(0, len(current_level), 2):

    left = current_level[i]

    right = current_level[i + 1]

    print("\nPair:")

    print("Left :", left)

    print("Right:", right)

    # Combine two items

    combined = left + right

    print("Combined:")

    print(combined)

    # SHA-256 hash

    hashed = sha256(combined)

    print("SHA-256:")

    print(hashed)

    # Store hash for next level

    new_level.append(hashed)

# STEP 5: Move to the next level

current_level = new_level

print("\n===== NEXT LEVEL =====")

```

```

    for i, item in enumerate(current_level, 1):

        print(f"Hash {i}: {item}")

    step += 1

# STEP 6: Only one hash remains

merkle_root = current_level[0]

print("\n===== FINAL STEP =====")

print("Merkle Root:")

print(merkle_root)

return merkle_root

# =====

# EXAMPLE

# =====

transactions = [

    "Transaction 1",

    "Transaction 2",

    "Transaction 3",

    "Transaction 4",

    "Transaction 5"

]

# Build Merkle Tree

merkle_root = build_merkle_tree(transactions)

# Final output

print("\n=====")

print("FINAL MERKLE ROOT:")

print(merkle_root)

print("=====")

```

Output:

===== STEP 1: ORIGINAL TRANSACTIONS =====

Transaction 1: Transaction 1

Transaction 2: Transaction 2

Transaction 3: Transaction 3

Transaction 4: Transaction 4

Transaction 5: Transaction 5

===== STEP 2: CURRENT LEVEL =====

1. Transaction 1

2. Transaction 2

3. Transaction 3

4. Transaction 4

5. Transaction 5

Number of items is ODD.

Duplicating the last item...

Duplicated: Transaction 5

===== HASHING PAIRS =====

Pair:

Left : Transaction 1

Right: Transaction 2

Combined:

Transaction 1Transaction 2

SHA-256:

39704f929d837dc8bd8e86c70c4fb06cf740e7294f1036d030e92fe545f18275

Pair:

Left : Transaction 3

Right: Transaction 4

Combined:

Transaction 3Transaction 4

SHA-256:

64833afa7026409be938e6e21a643749233e5d418b906fe5b6f304e7a7636eef

Pair:

Left : Transaction 5

Right: Transaction 5

Combined:

Transaction 5Transaction 5

SHA-256:

028a72e6c29761975bdef86d833b90f19823a90c8af6a59ed660d81b302f6015

===== NEXT LEVEL =====

Hash 1: 39704f929d837dc8bd8e86c70c4fb06cf740e7294f1036d030e92fe545f18275

Hash 2: 64833afa7026409be938e6e21a643749233e5d418b906fe5b6f304e7a7636eef

Hash 3: 028a72e6c29761975bdef86d833b90f19823a90c8af6a59ed660d81b302f6015

===== STEP 3: CURRENT LEVEL =====

1. 39704f929d837dc8bd8e86c70c4fb06cf740e7294f1036d030e92fe545f18275

2. 64833afa7026409be938e6e21a643749233e5d418b906fe5b6f304e7a7636eef

3. 028a72e6c29761975bdef86d833b90f19823a90c8af6a59ed660d81b302f6015

Number of items is ODD.

Duplicating the last item...

Duplicated: 028a72e6c29761975bdef86d833b90f19823a90c8af6a59ed660d81b302f6015

===== HASHING PAIRS =====

Pair:

Left : 39704f929d837dc8bd8e86c70c4fb06cf740e7294f1036d030e92fe545f18275

Right: 64833afa7026409be938e6e21a643749233e5d418b906fe5b6f304e7a7636eef

Combined:

39704f929d837dc8bd8e86c70c4fb06cf740e7294f1036d030e92fe545f1827564833afa7026409be
938e6e21a643749233e5d418b906fe5b6f304e7a7636eef

SHA-256:

0bc1c5cf4cc8f4915cdf888eca02682416c6be663d7706b9fb0933038ab9981a

Pair:

Left : 028a72e6c29761975bdef86d833b90f19823a90c8af6a59ed660d81b302f6015

Right: 028a72e6c29761975bdef86d833b90f19823a90c8af6a59ed660d81b302f6015

Combined:

028a72e6c29761975bdef86d833b90f19823a90c8af6a59ed660d81b302f6015028a72e6c2976197
5bdef86d833b90f19823a90c8af6a59ed660d81b302f6015

SHA-256:

907e9391a864ae81a234afc4a55bcad3e03cf067e08fbdabb46a13a66804c1b

===== NEXT LEVEL =====

Hash 1: 0bc1c5cf4cc8f4915cdf888eca02682416c6be663d7706b9fb0933038ab9981a

Hash 2: 907e9391a864ae81a234afc4a55bcad3e03cf067e08fbdabb46a13a66804c1b

===== STEP 4: CURRENT LEVEL =====

1. 0bc1c5cf4cc8f4915cdf888eca02682416c6be663d7706b9fb0933038ab9981a

2. 907e9391a864ae81a234afc4a55bcad3e03cf067e08fbdabb46a13a66804c1b

===== HASHING PAIRS =====

Pair:

Left : 0bc1c5cf4cc8f4915cdf888eca02682416c6be663d7706b9fb0933038ab9981a

Right: 907e9391a864ae81a234afc4a55bcad3e03cf067e08fbdabb46a13a66804c1b

Combined:

0bc1c5cf4cc8f4915cdf888eca02682416c6be663d7706b9fb0933038ab9981a907e9391a864ae81a
234afc4a55bcad3e03cf067e08fbdabb46a13a66804c1b

SHA-256:

a4a18941de1162b17a46c4f8c87d8a0850b46fad17ac881340061d9233785077

===== NEXT LEVEL =====

Hash 1: a4a18941de1162b17a46c4f8c87d8a0850b46fad17ac881340061d9233785077

===== FINAL STEP =====

Merkle Root:

a4a18941de1162b17a46c4f8c87d8a0850b46fad17ac881340061d9233785077

=====

FINAL MERKLE ROOT:

a4a18941de1162b17a46c4f8c87d8a0850b46fad17ac881340061d9233785077

=====

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.